



Computational thinking and AI  
计算思维与人工智能  
兴趣班

以法十一学校 中国科学院计算所  
龙溪实验中学

<https://ai-class-shiyilongyue.github.io/>

# 十一、计算机如何创造图像

---

中国科学院计算技术研究所

邢云冰

2026.06.03

# 授课老师简介



**邢云冰**

中国科学院计算所  
高级工程师

研究兴趣包括：数字人手语、人机交互与数字内容生成。

## 研究成果：

长期深耕人工智能模型架构设计、工程化实现与应用落地，在手语合成、人机交互及数字内容生成领域拥有深厚的技术积淀。

主导研发的冬奥手语播报数字人系统、气象节目的手语字幕系统、爱心小屋远程亲情互动系统、残疾人远程手语翻译服务平台、科教卫同屏互动服务平台等广泛应用于残疾人信息无障碍公共服务领域，打通了AI算法模型从实验室原型到规模化应用的闭环。

已积累发明专利30余项（已授权17项，已转让5项）。

## 获奖荣誉：

CCF科学技术发明一等奖

北京市科学技术奖二等奖（两次）

中国产学研合作创新成果奖

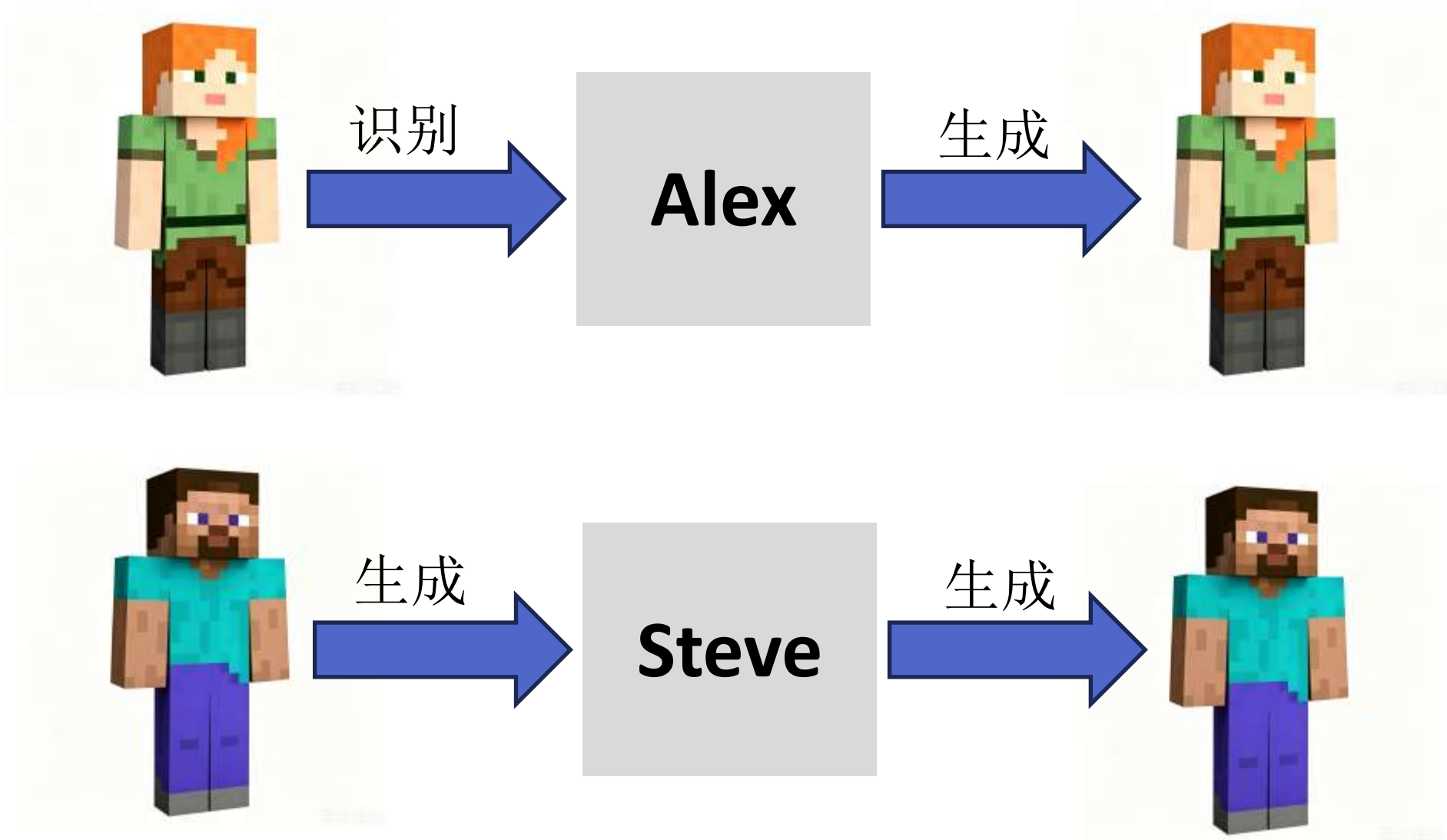
## CNN手写数字识别器

识别器本质上是一个降维与压缩的过程，它将  $28 \times 28$  像素的高维空间映射到了一个包含 10 个类别的低维离散空间。

它能告诉我们“这是什么”，但无法描述“这长什么样”。

| 组件   | 功能描述                           |
|------|--------------------------------|
| 输入   | 接收 $28 \times 28$ 的灰度图像（784像素） |
| 卷积层  | 提取特征(边缘、线条、形状、语义)              |
| 池化层  | 下采样主要特征，稀疏处理                   |
| 全连接层 | 整合全局特征，降维至 10 维                |
| 输出   | Softmax 计算类别概率分布               |

# 计算机生成图像



# 身边的识别和生成



裁判员



人脸识别



地图导航



相册分类



批改作业



创作者



语音聊天



生成提纲



文本聊天



美术创作

# 逆向思维：如何生成一张图像

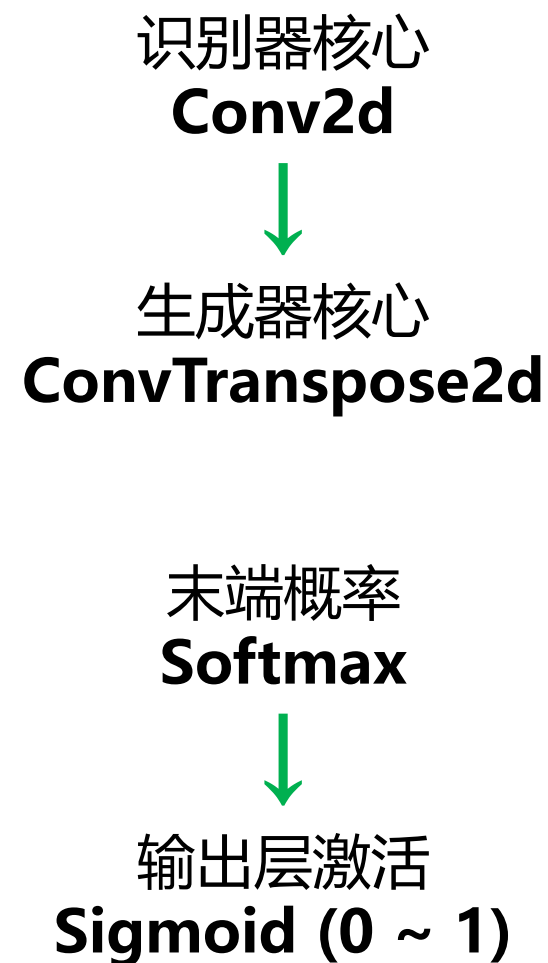
## 结构反转

这种逆向结构构成了生成器的基础，它尝试从一个简单的“指令”出发，逐步构建出复杂的视觉细节。

## 算子替换

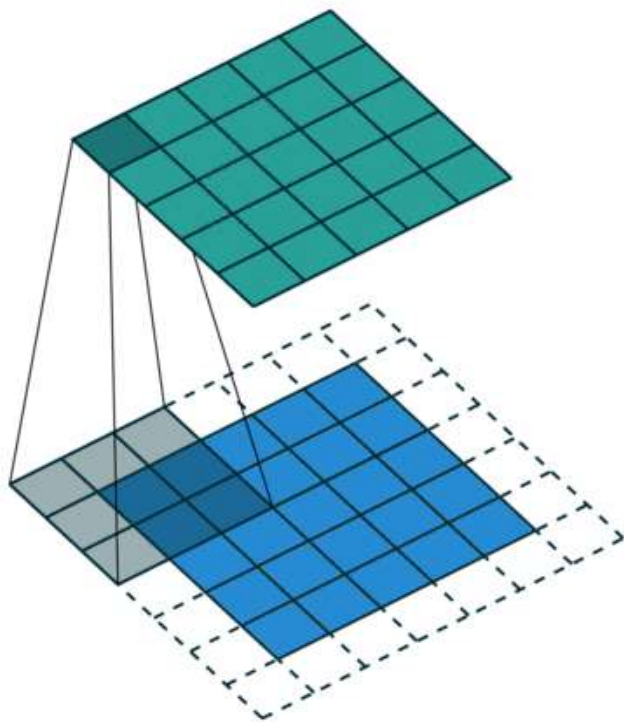
将传统的卷积替换为转置卷积，通过在输入像素间填充零并应用卷积核，实现了特征图的上采样与尺寸放大。

将末端的 Softmax 替换为 Sigmoid，将图像像素值归一化在 0 到 1 之间，Sigmoid 的输出特性匹配了像素值的范围。

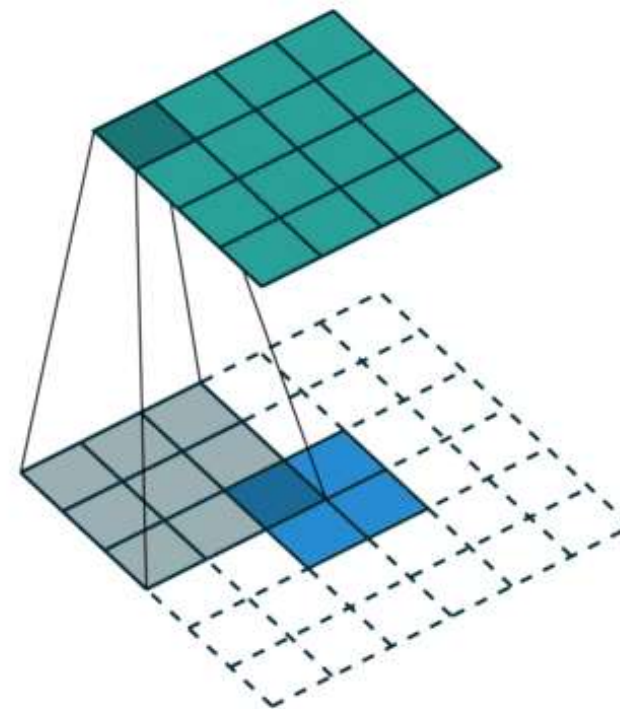




# Conv2d Vs ConvTranspose2d



$$\text{out\_size} = (\text{in\_size} + 2 * \text{padding} - (\text{kernel\_size} - 1)) / \text{stride}$$



$$\text{out\_size} = (\text{in\_size} - 1) * \text{stride} - 2 * \text{padding} + \text{kernel\_size}$$

## 1 计算均值和方差

对于每个通道，独立计算这批输入图像中所有像素的平均值 $\mu$ 和方差 $\sigma^2$

## 2 归一化

对于每个通道的每个像素，减去该通道对应的均值 $\mu$ 并除以标准差 $\sigma$ ，使其符合均值为0，方差为1的分布

## 3 缩放和平移

引入可学习的参数 $\gamma$ 和 $\beta$ ，让网络可以学习恢复数据的原始表达能力



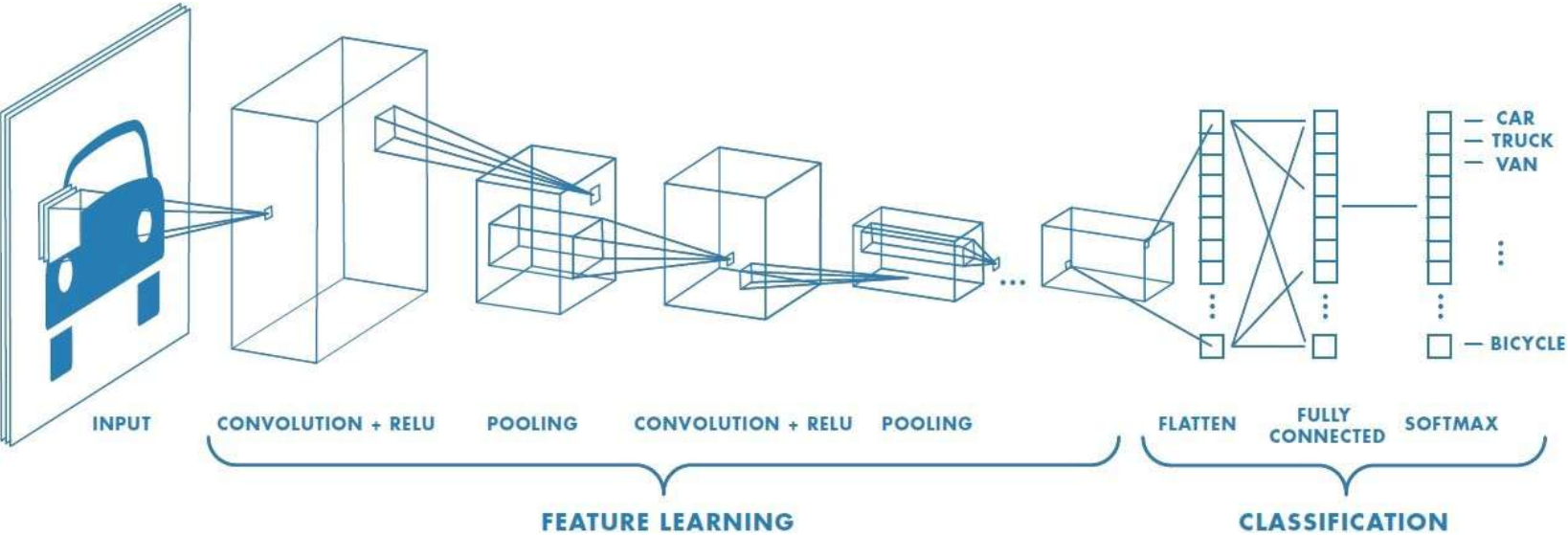
# BatchNorm2d



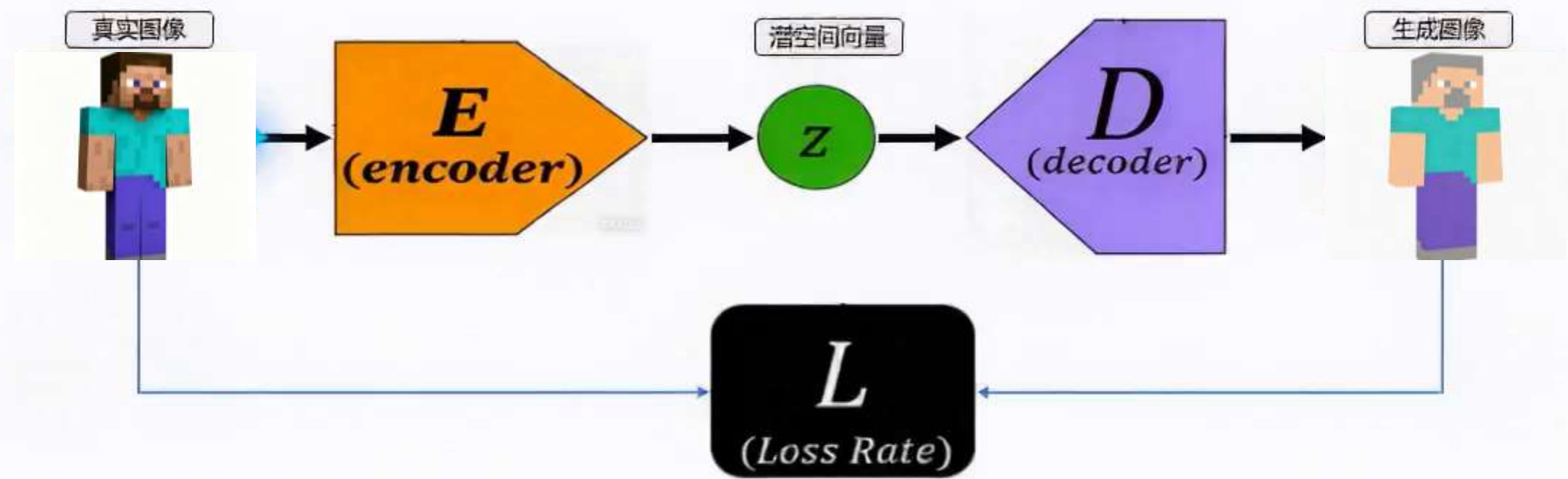
Computational thinking and AI  
计算思维与人工智能  
兴趣班



# 变分自编码器 (VAE)



# 变分自编码器 (VAE)



| 阶段   | 流程描述               | 核心目标           |
|------|--------------------|----------------|
| 编码阶段 | 输入图像 → 编码器 → 潜空间向量 | 提取图像的本质特征      |
| 解码阶段 | 空间向量 → 解码器 → 重构图像  | 还原原始图像的视觉细节    |
| 训练目标 | 最小化输入与输出之间的重构误差    | 实现 “输入什么，输出什么” |

# pytorch实战：构建编码器

```
33 # 编码器
34 class Encoder(nn.Module):
35     def __init__(self, in_channels=1, out_channels=100):
36         super(Encoder, self).__init__()
37         # out_size = (in_size + 2*padding - (kernel_size-1)) / stride
38         self.conv1 = nn.Conv2d(in_channels=in_channels, out_channels=32, kernel_size=4, stride=2, padding=1) # 1x28x28 -> 64x14x14
39         self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=4, stride=2, padding=1) # 64x14x14 -> 128x7x7
40         self.conv3 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=3, stride=2, padding=1) # 128x7x7 -> 256x4x4
41         self.conv4 = nn.Conv2d(in_channels=128, out_channels=out_channels, kernel_size=4) # 256x4x4 -> 100x1x1
42         self.bn1 = nn.BatchNorm2d(32)
43         self.bn2 = nn.BatchNorm2d(64)
44         self.bn3 = nn.BatchNorm2d(128)
45         self.relu = nn.ReLU(True)
46         self.identity = nn.Identity()
47
48     def forward(self, x):
49         x = self.relu(self.bn1(self.conv1(x))) # 28x28 -> 14x14
50         x = self.relu(self.bn2(self.conv2(x))) # 14x14 -> 7x7
51         x = self.relu(self.bn3(self.conv3(x))) # 7x7 -> 4x4
52         x = self.conv4(x) # 4x4 -> 1x1
53         x = x.view(x.size(0), -1) # 100*1*1 -> 100
54         x = self.identity(x)
55         return x
```



# pytorch实战：解码器

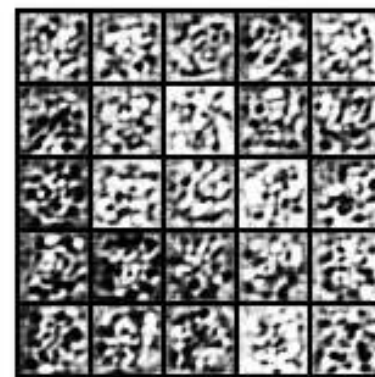
```
57 # 解码器
58 class Decoder(nn.Module):
59     def __init__(self, in_channels=100, out_channels=1):
60         super(Decoder, self).__init__()
61         # out_size = (in_size - 1) * stride - 2*padding + kernel_size
62         self.conv1 = nn.ConvTranspose2d(in_channels=in_channels, out_channels=128, kernel_size=4) # 100x1x1 -> 128x4x4
63         self.conv2 = nn.ConvTranspose2d(in_channels=128, out_channels=64, kernel_size=3, stride=2, padding=1) # 128x4x4 -> 64x7x7
64         self.conv3 = nn.ConvTranspose2d(in_channels=64, out_channels=32, kernel_size=4, stride=2, padding=1) # 64x7x7 -> 32x14x14
65         self.conv4 = nn.ConvTranspose2d(in_channels=32, out_channels=out_channels, kernel_size=4, stride=2, padding=1) # 32x14x14 -> 1x28x28
66         self.bn1 = nn.BatchNorm2d(128)
67         self.bn2 = nn.BatchNorm2d(64)
68         self.bn3 = nn.BatchNorm2d(32)
69         self.relu = nn.ReLU(True)
70         self.sigmoid = nn.Sigmoid()
71
72     def forward(self, x):
73         x = x.view(x.size(0), -1, 1, 1) # 100*1*1 -> 100x1x1
74         x = self.relu(self.bn1(self.conv1(x))) # 1x1 -> 4x4
75         x = self.relu(self.bn2(self.conv2(x))) # 4x4 -> 7x7
76         x = self.relu(self.bn3(self.conv3(x))) # 7x7 -> 14x14
77         x = self.conv4(x) # 14x14 -> 28x28
78         x = self.sigmoid(x)
79         return x
```

# 实验结果

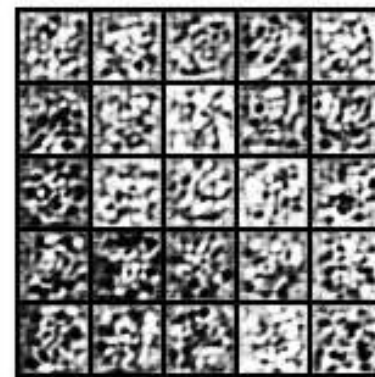
## 现象

训练完成后，仅使用解码器并输入随机向量，输出往往是模糊的块状物或毫无意义的“乱码”，而非清晰的数字。

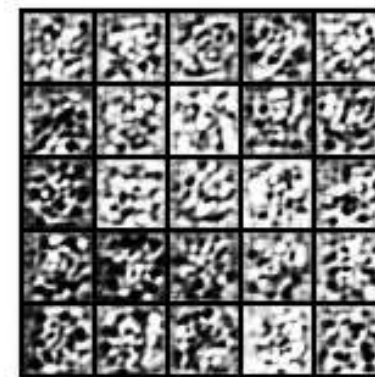
这种现象揭示了深度学习中一个深刻的问题：潜空间向量的分布失衡。



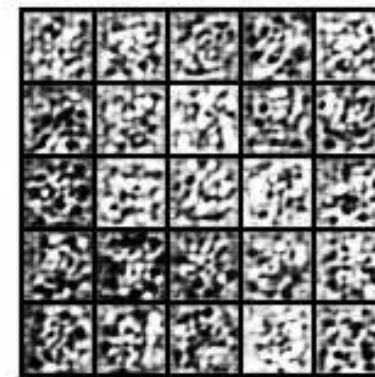
epoch\_22.png



epoch\_28.png



epoch\_23.png



epoch\_29.png



## 思考

- 1 训练时，编码器输出的向量始终对应于“正常的”数字图像。
- 2 潜空间（Latent Space）极大，正常的嵌入向量实际上仅占据了其中极小的一部分。
- 3 随机生成的向量大概率落在解码器从未见过的“非正常”区域。

根本原因：VAE 默认所有输入的图像都是正常的，它只学会了如何“复刻”正常数据，却从未学习过如何拒绝或识别“不正常”的输入。

# 损失函数的缺陷

| 模型类型   | 奖励机制   | 惩罚机制       | 潜在风险        |
|--------|--------|------------|-------------|
| 识别器    | 正确分类奖励 | 错误分类惩罚     | 无法识别“非数字”输入 |
| VAE生成器 | 像素重构奖励 | 缺乏对异常输入的惩罚 | 随机输入导致输出乱码  |

原因：VAE 仅关注重构，即对正常的图像给予奖励；但对于**不正常的**输入，损失函数没有提供足够的惩罚。

解决思路：不仅要让生成器学会像什么，还要引入机制惩罚那些**不像**的作品。

# 解决方法：引入对抗机制

生成器  
Generator

VS

鉴别器  
Discriminator

从随机噪声中创造图像。  
核心目标：“以假乱真”，不断提升创作质量以骗过鉴别器

判断图像的来源（真或假）。  
核心目标：“去伪存真”，建立严苛的真假判断标准。

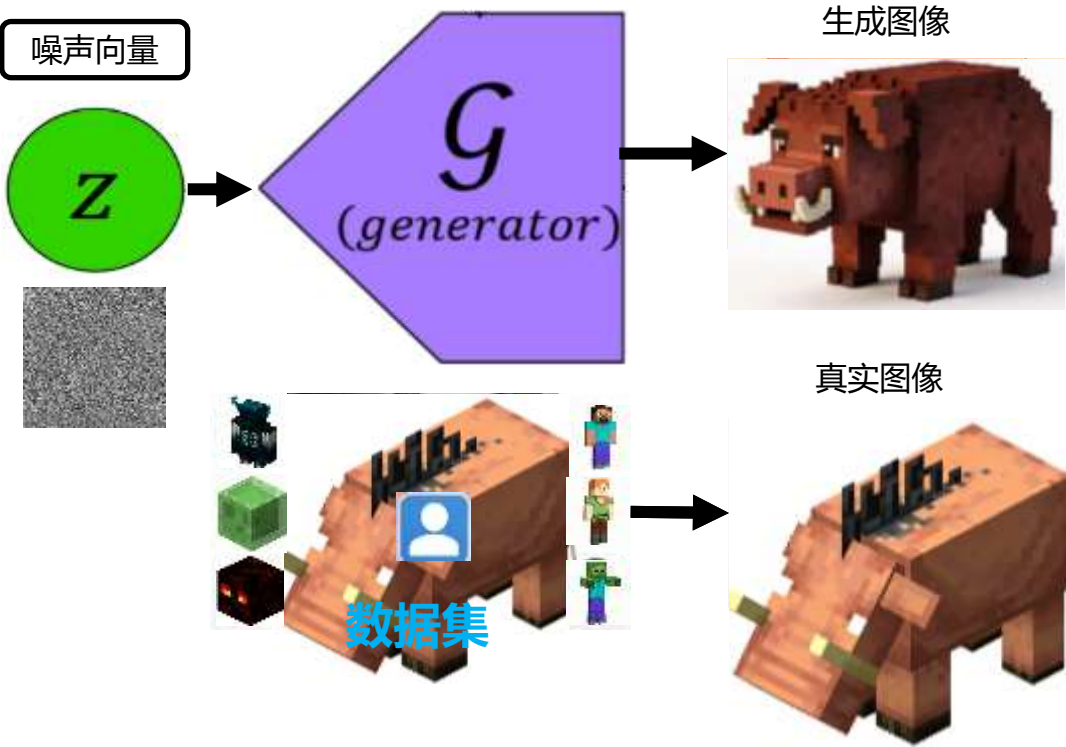
改造方法：将原识别器的最后一层修改为 *Sigmoid* 激活函数，使其输出从“类别特征”转变为“真实性概率”（0 代表假，1 代表真）。

# 生成对抗网络 (GAN)

噪声向量

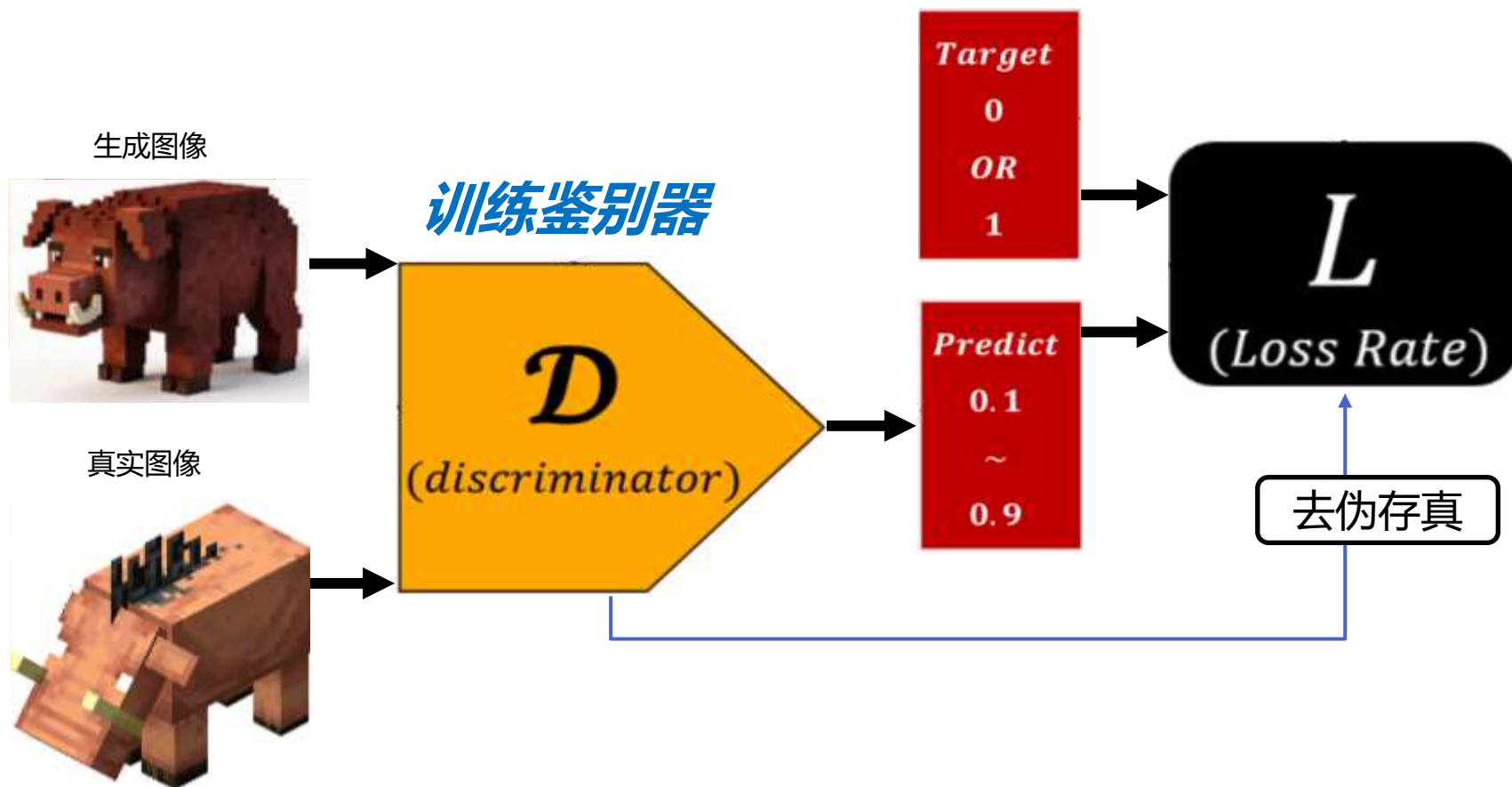


# 生成对抗网络 (GAN)

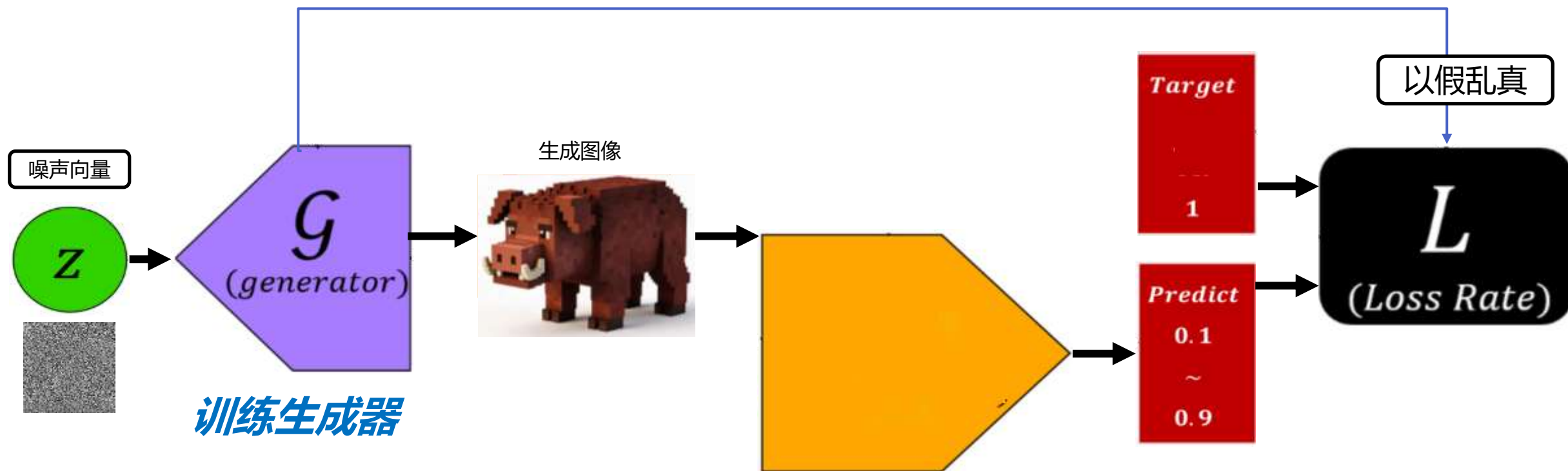




# 生成对抗网络 (GAN)

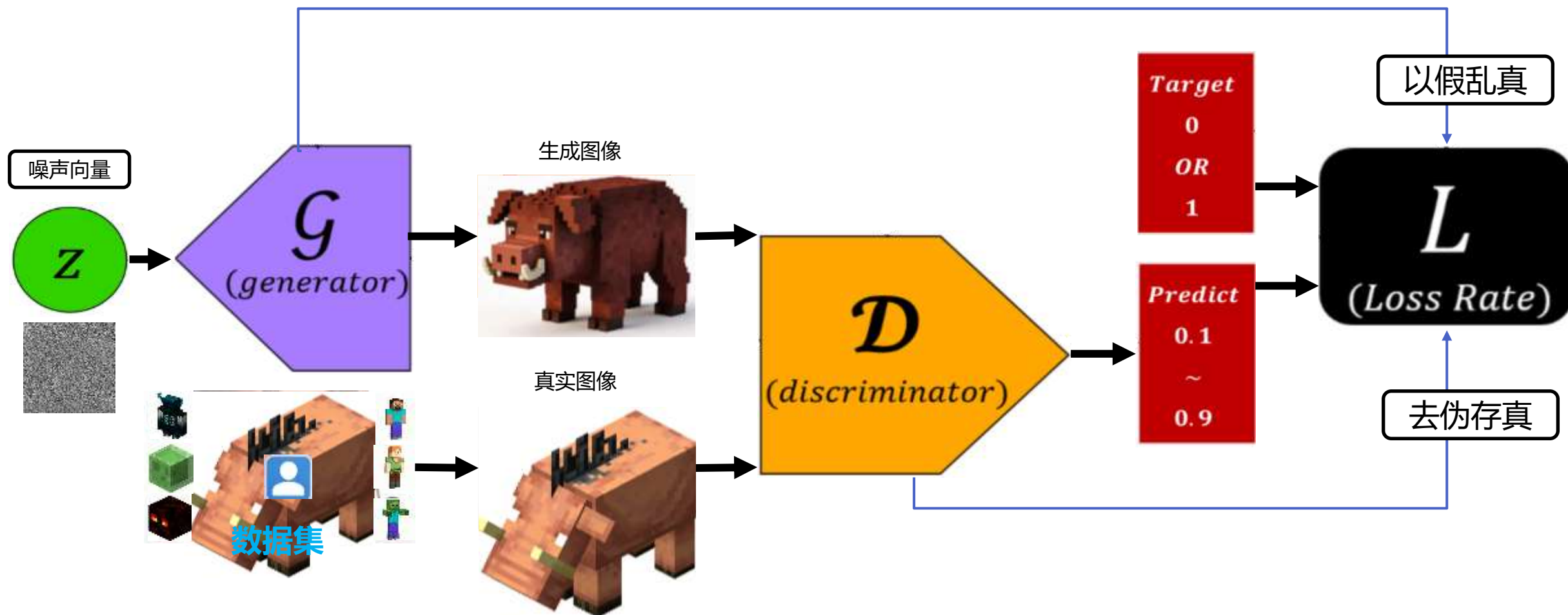


# 生成对抗网络 (GAN)



# 生成对抗网络 (GAN)

## Generative Adversarial Network



# GAN 的训练技巧：博弈

## 1 训练鉴别器 (Discriminator)

目标是最大化识别准确率。最大化  $D(x) + (1 - D(G(z))) == 2$

## 2 训练生成器 (Generator)

目标是最小化鉴别器的判断精度，即“骗过”鉴别器。最大化  $D(G(z)) == 1$

## 3 交替博弈

这种“左右互搏”的训练方式，使得损失函数包含了互相矛盾的两个方面。生成器在不断的失败中学习到了真实数字的统计规律。

# 博弈论的起源：纳什

## 规则

每个学生在纸上写下一个0到100之间的整数和姓名，不能讨论，也不能偷看别人写什么。所有人写完后，助教把全班的数字加起来，算出平均数，然后**取这个平均数的一半**，谁写的数字最接近“平均数的一半”，谁就获胜。

思路：别人怎么选，别人觉得你会怎么选





# 课堂提问：训练模式崩溃

1 鉴别器 太弱

2 鉴别器 太强

3 生成器 太弱

# 课堂提问：训练模式崩溃

## 1 鉴别器 太弱

生成器也变得懒惰，敷衍了事，比烂

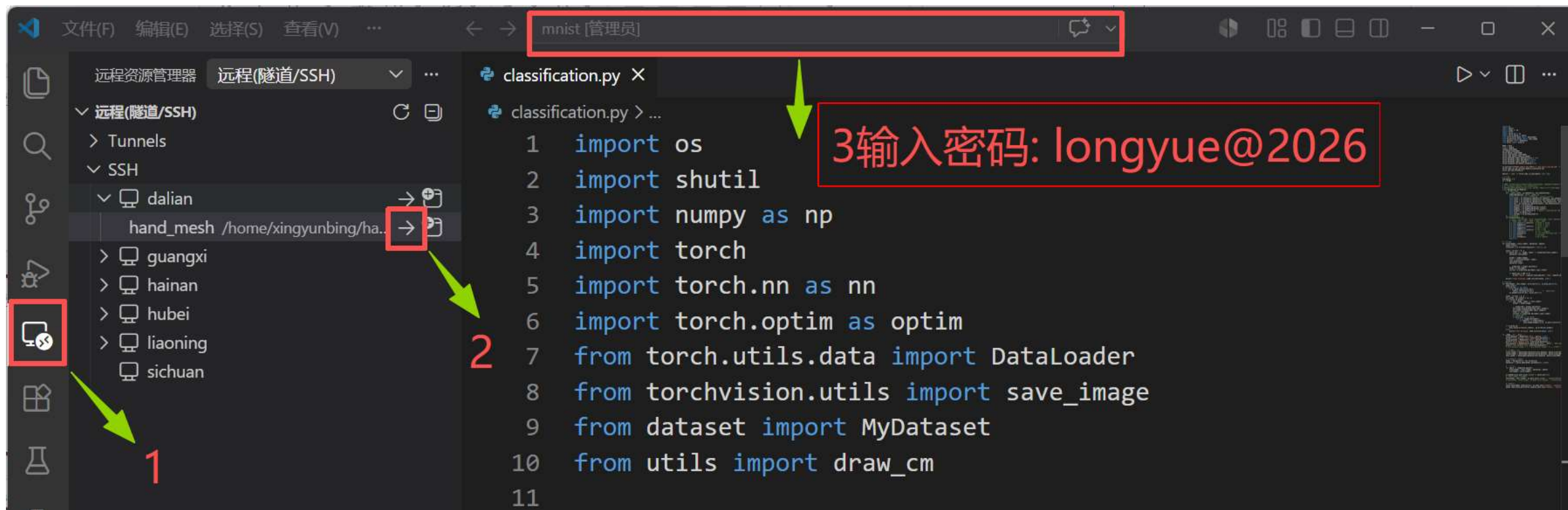
## 2 鉴别器 太强

生成器信心受挫，撂挑子不玩了或样式单一

## 3 生成器 太弱

只要鉴别器设计正常，还可以继续抢救

# pytorch实战：打开远程服务器



# pytorch实战：构建鉴别器



Computational thinking and AI  
计算思维与人工智能  
兴趣班

```
3 # 定义鉴别器
4 class Discriminator(nn.Module):
5     def __init__(self, in_channels=1, out_channels=1):
6         super(Discriminator, self).__init__()
7         # out_size = (in_size + 2*padding - (kernel_size-1)) / stride
8         self.conv1 = nn.Conv2d(in_channels, out_channels=64, kernel_size=4, stride=2, padding=1) # 1x28x28 -> 64x14x14
9         self.conv2 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=4, stride=2, padding=1) # 64x14x14 -> 128x7x7
10        self.conv3 = nn.Conv2d(in_channels=128, out_channels=256, kernel_size=3, stride=2, padding=1) # 128x7x7 -> 256x4x4
11        self.conv4 = nn.Conv2d(in_channels=256, out_channels=out_channels, kernel_size=4) # 256x4x4 -> 1x1x1
12        self.bn1 = nn.BatchNorm2d(64)
13        self.bn2 = nn.BatchNorm2d(128)
14        self.bn3 = nn.BatchNorm2d(256)
15        self.relu = nn.ReLU(True)
16        if out_channels == 1:
17            self.lastlayer = nn.Sigmoid() # 鉴别器输出1类, 使用Sigmoid激活函数
18        elif out_channels == 10:
19            self.lastlayer = nn.Softmax(1) # 识别器输出10类, 使用Softmax激活函数
20        else:
21            self.lastlayer = nn.Identity() # 编码器输出潜空间嵌入向量, 不使用激活函数
22
23    def forward(self, x):
24        x = self.relu(self.bn1(self.conv1(x))) # 28x28 -> 14x14
25        x = self.relu(self.bn2(self.conv2(x))) # 14x14 -> 7x7
26        x = self.relu(self.bn3(self.conv3(x))) # 7x7 -> 4x4
27        x = self.conv4(x) # 4x4 -> 1x1
28        x = x.view(x.size(0), -1) # 1*1*1 -> 1
29        x = self.lastlayer(x)
30        return x
```

# pytorch实战：构建生成器



Computational thinking and AI  
计算思维与人工智能  
兴趣班

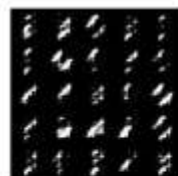
```
32 # 定义生成器
33 class Generator(nn.Module):
34     def __init__(self, in_channels=100, out_channels=1):
35         super(Generator, self).__init__()
36         # out_size = (in_size - 1) * stride - 2*padding + kernel_size
37         self.conv1 = nn.ConvTranspose2d(in_channels=in_channels, out_channels=256, kernel_size=4) # 100x1x1 -> 256x4x4
38         self.conv2 = nn.ConvTranspose2d(in_channels=256, out_channels=128, kernel_size=3, stride=2, padding=1) # 256x4x4 -> 128x7x7
39         self.conv3 = nn.ConvTranspose2d(in_channels=128, out_channels=64, kernel_size=4, stride=2, padding=1) # 128x7x7 -> 64x14x14
40         self.conv4 = nn.ConvTranspose2d(in_channels=64, out_channels=out_channels, kernel_size=4, stride=2, padding=1) # 64x14x14 -> 1x28x28
41         self.bn1 = nn.BatchNorm2d(256)
42         self.bn2 = nn.BatchNorm2d(128)
43         self.bn3 = nn.BatchNorm2d(64)
44         self.relu = nn.ReLU(True)
45         self.sigmoid = nn.Sigmoid()
46
47     def forward(self, x):
48         x = x.view(x.size(0), -1, 1, 1) # 100 -> 100x1x1
49         x = self.relu(self.bn1(self.conv1(x))) # 1x1 -> 4x4
50         x = self.relu(self.bn2(self.conv2(x))) # 4x4 -> 7x7
51         x = self.relu(self.bn3(self.conv3(x))) # 7x7 -> 14x14
52         x = self.sigmoid(self.conv4(x)) # 14x14 -> 28x28
53         return x
```



# 实验结果



Computational thinking and AI  
计算思维与人工智能  
兴趣班



epoch\_0.png



epoch\_1.png



epoch\_2.png



epoch\_3.png



epoch\_4.png



epoch\_5.png



epoch\_6.png



epoch\_7.png



epoch\_8.png



epoch\_9.png



epoch\_10.png



epoch\_11.png



epoch\_12.png



epoch\_13.png



epoch\_14.png



epoch\_15.png



epoch\_16.png



epoch\_17.png



epoch\_18.png



epoch\_19.png



epoch\_20.png



epoch\_21.png



epoch\_22.png



epoch\_23.png



epoch\_24.png



epoch\_25.png



epoch\_26.png



epoch\_27.png



epoch\_28.png



epoch\_29.png

# 实验时间



Computational thinking and AI  
计算思维与人工智能  
兴趣班

```
3 # 定义鉴别器
4 class Discriminator(nn.Module):
5     def __init__(self, in_channels=1, out_channels=1):
6         super(Discriminator, self).__init__()
7         # out_size = (in_size + 2*padding - (kernel_size-1)) / stride
8         self.conv1 = nn.Conv2d(in_channels, out_channels=32, kernel_size=4, stride=2, padding=1) # 1x28x28 -> 14x14
9         self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=4, stride=2, padding=1) # 14x14 -> 7x7
10        self.conv3 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=3, stride=2, padding=1) # 7x7 -> 4x4
11        self.conv4 = nn.Conv2d(in_channels=128, out_channels=out_channels, kernel_size=4) # 4x4 -> 1x1
12        self.bn1 = nn.BatchNorm2d(32)
13        self.bn2 = nn.BatchNorm2d(64)
14        self.bn3 = nn.BatchNorm2d(128)
15        self.relu = nn.ReLU(True)
16        if out_channels == 1:
17            self.lastlayer = nn.Sigmoid() # 鉴别器输出1类, 使用Sigmoid激活函数
18        elif out_channels == 10:
19            self.lastlayer = nn.Softmax(1) # 鉴别器输出10类, 使用Softmax激活函数
20        else:
21            self.lastlayer = nn.Identity() # 编码器输出潜空间嵌入向量, 不使用激活函数
22
23    def forward(self, x):
24        x = self.relu(self.bn1(self.conv1(x))) # 28x28 -> 14x14
25        x = self.relu(self.bn2(self.conv2(x))) # 14x14 -> 7x7
26        x = self.relu(self.bn3(self.conv3(x))) # 7x7 -> 4x4
27        x = self.conv4(x) # 4x4 -> 1x1
28        x = x.view(x.size(0), -1) # 1*1*1 -> 1
29        x = self.lastlayer(x)
30        return x
31
```

Epoch: [2/30], Step: [100/500], D\_Loss: 0.0015, G\_Loss: 9.4745  
Epoch: [2/30], Step: [200/500], D\_Loss: 0.0006, G\_Loss: 9.9827  
Epoch: [2/30], Step: [300/500], D\_Loss: 0.0030, G\_Loss: 9.4686  
Epoch: [2/30], Step: [400/500], D\_Loss: 0.0013, G\_Loss: 8.1501

# 经典的图像识别网络

| 时间   | 名称        | 层数  | 机构/作者              | 特点                                 |
|------|-----------|-----|--------------------|------------------------------------|
| 1998 | LeNet     | 6   | 贝尔实验室/Yann LeCun   | 第一个卷积网络，卷积、激活、池化和全连接               |
| 2012 | AlexNet   | 8   | 多伦多大学/Alex, Hinton | ImageNet Top5错误率15.3%，非深度学习方法26.2% |
| 2014 | VGG       | 19  | 牛津大学               | ImageNet Top5错误率减小到7.3%            |
| 2014 | GoogLeNet | 22  | Google             | ImageNet Top5错误率6.7%               |
| 2015 | ResNet    | 152 | 微软亚洲研究院/何恺明        | ImageNet Top5错误率3.57%              |
| 2016 | ResNext   |     | Facebook/谢赛宁, 何恺明  | ImageNet Top5错误率3.03%              |
| 2016 | DenseNet  | 264 | 康奈尔大学, Facebook    | ImageNet Top5错误率5.29%              |
| 2017 | SENet     | 154 | Momenta/胡杰         | ImageNet Top5错误率2.25%              |

ImageNet2017年停赛，从最初的算法对物体进行识别的准确率只有71.8%上升到现在的97.3%，识别错误率已经远远低于人类的5.1%。

# 主流的图像/视频生成模型

| 时间        | 名称                    | 规模(亿)   | 机构/公司          | 特点                    |
|-----------|-----------------------|---------|----------------|-----------------------|
| 2022      | Midjourney            |         | Midjourney Inc | 艺术创作，缺乏对话式编辑功能        |
| 2022/2023 | StableDiffusion       |         | Stability AI   | 开源，支持ControlNet       |
| 2026      | NanoBanana2           |         | Google         | 将“世界知识”融入生成，生成内容事实准确性 |
| 2023      | StableDiffusion Video |         | Stability AI   | 开源的基础模型，默认无音频，时长受限    |
| 2024/2026 | Sora1/2               |         | OpenAI         | 能处理复杂场景和叙事            |
| 2025      | 混元                    | 130     | 腾讯             | 开源，对人物表情动作和镜头运动有专项增强  |
| 2026      | 通义万相 (Wan2)           | 140/270 | 阿里巴巴           | MoE架构，推理成本低，对电影风格控制精准 |
| 2026      | Seedance2             |         | 字节             | 双分支扩散架构，音画原生同步和物理世界建模 |



## • 告诉我如何做清蒸鲈鱼

清蒸鲈鱼的关键是：**鱼新鲜、腥味处理好、火候别过、最后浇热油激香。**

### 清蒸鲈鱼做法

#### 一、准备材料

鲈鱼 1 条，约 500–700 克，姜、葱、料酒、盐、蒸鱼豉油、食用油

可选：少量白胡椒粉、红椒丝点缀

#### 二、处理鲈鱼

1. 鲈鱼去鳞、去内脏、去鳃，清洗干净。 2. 特别要把鱼肚子**里的黑膜和血块**洗干净，这是腥味的重要来源。 3. 在鱼身两侧各划 2–3 刀，方便受热和入味。 4. 用少量盐、料酒、姜片抹一下鱼身和鱼肚，腌制 **10 分钟左右**。

注意：盐不要太多，后面还要加蒸鱼豉油。

#### 三、上锅蒸

1. 盘底铺几片姜和葱段，把鱼放上去。 2. 鱼肚里也塞一点姜葱。 3. 水烧开后再放鱼，大火蒸。 4. 根据鱼大小控制时间：

500 克左右：蒸 **7–8 分钟**

600–700 克：蒸 **9–10 分钟**

蒸好后不要马上开盖，可以关火焖 **1–2 分钟**。

#### 四、倒掉腥水



# AI的应用2：图像

- 将照片中的人物替换为卡通形象



- 生成一个我的世界风格的《呐喊》名画



# AI的应用3：代码

帮我生成一个“饥饿的毛毛虫”的小游戏（python），不用pygame和sys库

Thought for 4s >

**Hungry Caterpillar Python Game** · python 复制 编辑 下载 运行代码

```
"""
饥饿的毛毛虫小游戏 (Python / tkinter)

特点:
- 不使用 pygame
- 不使用 sys
- 只使用 Python 标准库

运行方式:
1. 保存为 hungry_caterpillar.py
2. 运行: python hungry_caterpillar.py

玩法:
- 方向键 / WASD 控制毛毛虫移动
- 吃苹果、草莓得分并变长
```



# AI的应用4：视频





# AI的应用5：声音



# AI犯错：看不清

- 特斯拉错把卡车当天空

2016 年，美国佛罗里达州一特斯拉轿车使用自动驾驶时，与一辆横穿道路的白色半挂卡车相撞，当时卡车拖挂车的白色侧面与明亮天空背景接近





# AI犯错：逻辑错误

## • 9.11与9.9哪个大



9.11 比 9.9 大。比较这两个数字的方法是直接看小数点后面的数字。因为 11 大于 9，所以 9.11 大

## • 逻辑混乱-洗车问题

我想去洗车，洗车店距离我家 50 米，你说我应该开车过去还是走过去？

既然洗车店离你家只有50米，走路去会更方便、环保而且省油。除非你有其他需要开车带走的东西，或者觉得天气不好、步行不舒服，否则走路是最轻松的选择。



# AI犯错：幻觉

- 一家知名学术打假机构曝光了一篇2026年4月刚发表的论文，作者声称他们开发了一个“机器学习驱动的蛋白质表达优化平台”，论文的很多参考文献要不根本就不存在，要不就是标题，作者，期刊完全驴唇不对马嘴。

discussions with colleagues at Shanghai Jiao Tong University and Mukhtar Auezov South Kazakhstan State University, Kazakhstan.

## References

1. Incir I, Kaplan Ö. *Escherichia coli* as a versatile cell factory: advances and challenges in recombinant protein production. *Protein Expr Purif.* 2024;219:106463. PMID:38479588  
[View Article](#) • [PubMed/NCBI](#) • [Google Scholar](#)
2. Singh PK, Sharma V, Patil PB. Bacterial toxins in cancer therapy: current status and future directions. *Biochim Biophys Acta Rev Cancer.* 2022;1877:188747.  
[View Article](#) • [Google Scholar](#)
3. Zhang Y, Zhang J, Wang S. Targeted toxin therapy for hematological malignancies: recent advances and clinical perspectives. *J Hematol Oncol.* 2023;16:45.  
[View Article](#) • [Google Scholar](#)
4. Chaudhary N, Weissman D, Whitehead KA. mRNA vaccines for infectious diseases: principles, delivery and clinical translation. *Nat Rev Drug Discov.* 2021;20(11):817–38. PMID:34433919  
[View Article](#) • [PubMed/NCBI](#) • [Google Scholar](#)

5 GH Foundation



Access through your organization

Purchase PDF

Patient Access



Biochimica et Biophysica Acta (BBA) -  
Reviews on Cancer

Volume 1877, Issue 4, July 2022, 188747



Review

## HOXA1, a breast cancer oncogene

Magali Belpaire<sup>a,1</sup>, Arnaud Taminiau<sup>a,1</sup>, Dirk Geerts<sup>b,2,3</sup>, René Rezsohazy<sup>a,2</sup>



Chen et al. 2022

# AI犯错：不符合物理规律

## • 大家来找错：图片





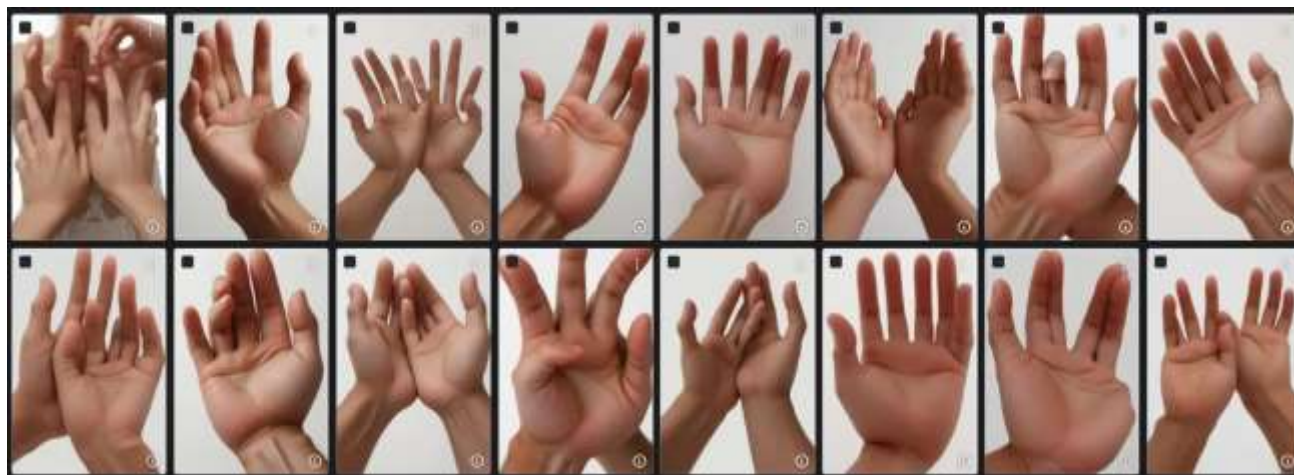
# AI犯错：不符合物理规律

## • 大家来找错：视频



# AI犯错：不符合物理规律

## • 难倒大模型的手





## AI像人，也可能看错、听错、判错

- 1 看不清：图片模糊、太亮、太暗、被遮挡
- 2 太相似：长得像，不代表就是
- 3 没见过：训练数据不够全面，遇到新情况容易懵
- 4 不理解：AI会看特征，但不一定懂场景，也没有情感和价值观

**AI的答案是“最可能”，不是“一定对”**

## • 被教坏的聊天机器人Tay

微软公司在推特社交平台上发布了名为Tay的人工智能聊天机器人，用户只需在推特上@她就能得到Tay的回复。不到24小时，她就被“教坏”了，成为一个集反犹太人、性别歧视、种族歧视于一身的“不良少女”。



# AI伦理： 伪造



Computational thinking and AI  
计算思维与人工智能  
兴趣班



# AI伦理：原创性

## 训练数据



✓ 包含受版权保护的作品

## 生成内容



归属权属于AI开发者



归属权属于原版权所有者



# AI伦理：偏见



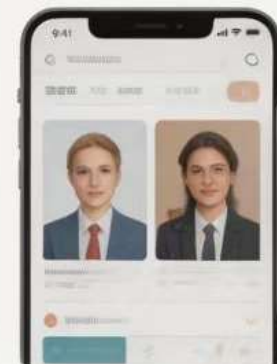
数据里：非裔头像标注「危险」，白人头像标安全」



数据里：女性只出现于「护士/秘书」，男性只出于「工程师/CEO」

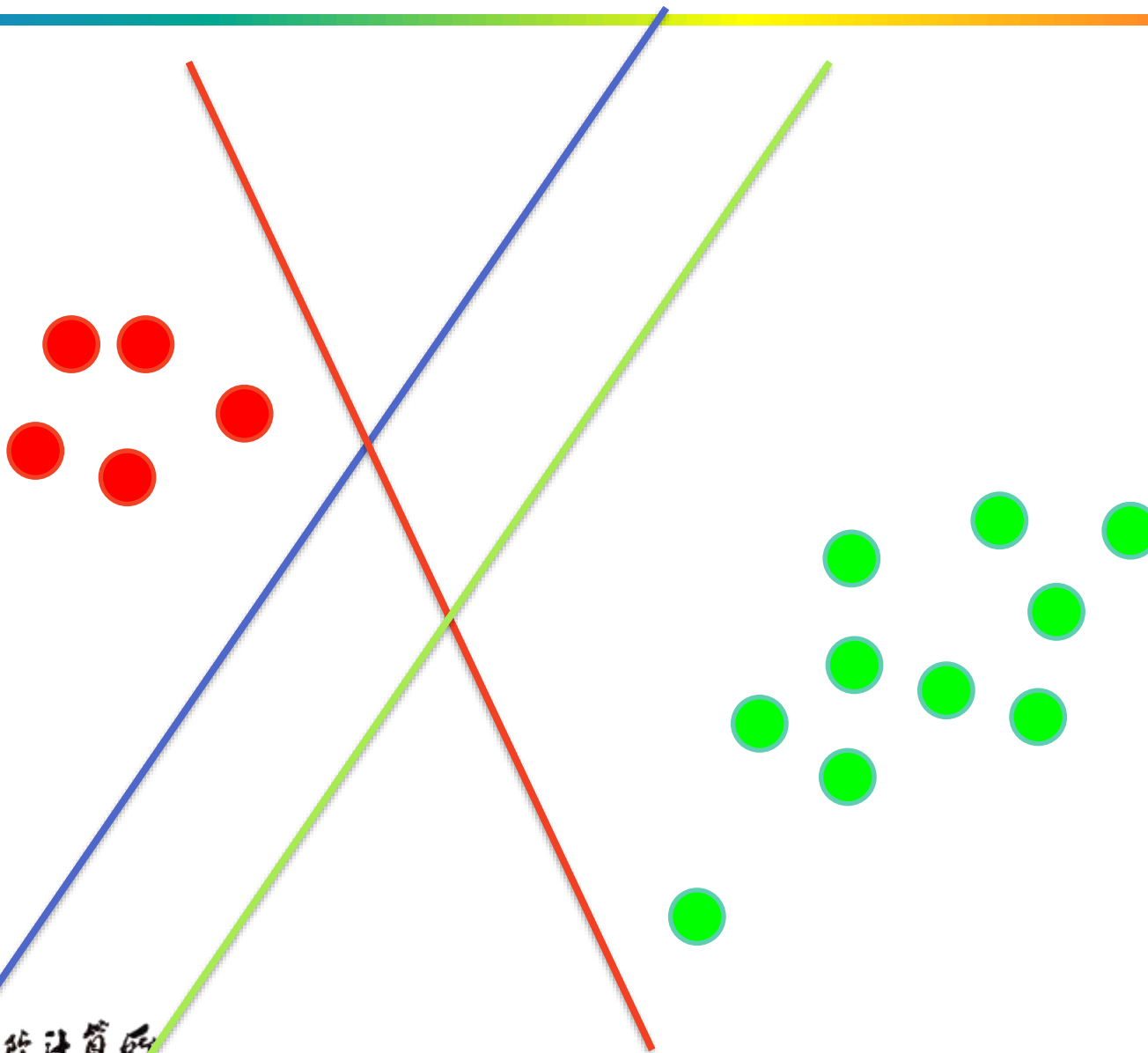


模型输出：误将非裔识别为「罪犯」



模型输出：给女性推荐「护士」，给男性推荐「工程师」

# 偏见的原因：非均衡



# AI伦理：生成式 AI 的双刃剑

## 真实性危机

### 深度伪造 (Deepfake)

技术可能被用于身份欺诈或虚假信息传播，挑战社会信任基础。

## 版权与归属

### 原创性界定

训练数据往往包含受版权保护的作品，生成内容的归属权尚存争议。

## 算法偏见

### 偏见放大

如果训练数据存在歧视，生成模型会无意识地放大这些社会偏见。

**技术本身是中立的，但它的应用方向取决于人类的选择。在追求‘以假乱真’的最高境界时，我们更应坚守‘求真务实’的伦理底线。**

# AI安全：提高警惕

## • 滥用AI的骗子可能冒充：

- 家人：假装遇到危险，要求赶快转账
- 老师/同学：发布假的语音通知
- 领导/客服：要求提供验证码、密码或付款
- 名人/公众人物：传播虚假讲话或假新闻

## • 遇到重要事情如涉及转账、验证码、密码、求救时，一定要做到：

- 停一下：不要立刻行动。
- 问细节：问只有本人知道的问题。
- 多渠道确认：挂掉电话，用原来的号码、微信或面对面确认。
- 设置家庭暗号：家人之间约定一个紧急确认口令。
- 不随便发语音：不要在网上随便公开自己的清晰语音素材。



- **判别式 AI 会“判断错”**

- 它可能把人认错、把物体看错、把正确答案判错，也可能把危险内容漏掉。
- AI 说“像”，不等于“就是”；AI 说“对”，也不等于“一定对”。

- **生成式 AI 会“编造”**

- 它可能编造不存在的知识、参考文献、法律案例，也可能生成看起来很真的假图片、假视频和假声音。
- AI 写得像真的，不代表是真的；AI 画得像照片，不代表是照片。

- **对AI输出的结果，特别是比较关键的内容要做到**

- 重要信息查来源：看是否来自官网、权威媒体、原始文件。
- 重要结论多方验证：不要只听一个AI的回答。
- 重要判断让人复核：尤其是人脸识别、医疗、法律、考试、转账等场景。
- 看到“太震惊”的内容先暂停：越离奇，越要先核实。

# 来自马斯克的总结



# 来自马斯克的总结



## 发现问题与原因

**现象：**为什么每次结果都不一样

**根源：**

- 1、随机初始化
- 2、并行计算（加法的顺序）
  - 精度丢失
  - 累积误差

```
import torch

val = torch.tensor(1.0e8, device='cuda')
print(val - 1)

val = torch.tensor(1000.0, device='cuda')
for _ in range(10000):
    val -= 0.1
print(val) # 预期0
```

**谢谢！ 请批评指正！**